PROJECT OVERVIEW

RunbookPilot

AI-Powered Runbook Assistant for Cloud Operations

Customer Problem Statement, Proposed Solution, Architecture, Technology Stack, and Deployment Scope

PROJECT	PRIMARY USERS	CORE USE CASE
RunbookPilot	SRE, DevOps, Cloud and Platform Engineers	Runbook retrieval, incident troubleshooting, evidence-backed operational guidance

Implementation-aligned technology baseline
FastAPI • LangGraph • Groq LLM • Hugging Face Sentence Transformers • Local persistent Qdrant • SQLite • Optional Tavily search • Docker

1. Problem Statement

Cloud and platform teams rely on runbooks, standard operating procedures, deployment rollback guides, and troubleshooting notes to resolve operational incidents. Even when documentation exists, engineers may spend valuable time locating the right instructions and deciding whether retrieved content applies to the current symptoms.

A basic retrieval-augmented generation (RAG) assistant can return loosely related text and still produce a confident answer. In incident response, guidance that is incomplete, irrelevant, or unsupported by source material can delay diagnosis or increase operational risk. Some questions may also need current public technical information that is not present in the local knowledge base.

Core challenge

Help engineers move from an operational question to practical next steps while making the answer traceable to retrieved documents, checking evidence quality, preserving conversation context, and using web search only when configured and needed.

2. Why a basic RAG flow is insufficient

Limitation	Impact
Retrieve once	The first search may miss relevant runbook content or return weak matches.
Generate without grading	The LLM may rely on incomplete or unrelated retrieved chunks.
No query refinement	A poorly phrased query can limit retrieval quality unless the workflow rewrites it.
No answer verification	The generated response needs checks for evidence support and usefulness.
No session continuity	Follow-up questions need prior incident context to avoid repeating the investigation.

3. Proposed Solution

RunbookPilot is an AI-powered runbook assistant built around a Self-RAG-inspired LangGraph workflow. It accepts a user question through a FastAPI endpoint, decides whether retrieval is needed, searches the local Qdrant knowledge base, grades retrieved evidence, and can rewrite the query or use optional Tavily search when internal evidence is insufficient. A Groq-hosted language model generates responses, while support and usefulness checks help guide refinement.

The application includes a browser-based interface, document upload and ingestion, source metadata, workflow trace information, SQLite persistence for LangGraph checkpoints, and SQLite audit logging. These are implemented components in the supplied project. Docker packaging is present; DigitalOcean deployment should be described as a target unless it has separately been completed and verified.

4. End-to-end workflow

Step	Workflow stage	What happens
1	Load conversation context	LangGraph restores checkpoint state for the active thread using SQLite persistence.
2	Contextualize question	Follow-up messages can be rewritten as standalone operational questions using prior conversation context.
3	Decide on retrieval	The workflow determines whether operational evidence is required or a direct response is appropriate.
4	Retrieve internal evidence	Sentence Transformer embeddings and local Qdrant similarity search retrieve the top-k document chunks.
5	Grade relevance	The LLM assesses whether retrieved chunks are relevant; weak results can trigger query rewriting and another retrieval attempt.
6	Optional web fallback	If configured and required, Tavily searches the web and returned evidence is assessed before use.
7	Generate and verify	The LLM drafts an evidence-based answer; support and usefulness checks can request revision within configured retry limits.
8	Return and persist	The API returns the answer, sources, route/trace metadata and saves audit information; checkpoint state supports follow-up turns.

Important implementation note

The workflow is Self-RAG-inspired rather than a claim of reproducing every detail of the original research paper. Structured-output model calls and provider compatibility must be tested with the configured Groq model before describing all verification branches as production-validated.

5. Example operational scenarios

Scenario A — Internal runbook has the answer

Question: “Our checkout API is returning 502 errors after a deployment. What should I check first?”

RunbookPilot searches the local knowledge base for relevant checkout and rollback guidance, grades the retrieved chunks, and uses the accepted context to generate recommended diagnostic steps. Source metadata can be returned so the engineer can inspect the supporting document. Web search should not be necessary when the internal evidence is sufficient.

Scenario B — Follow-up question

Follow-up: “What should I check next if that does not solve it?”

The same conversation thread can restore its LangGraph checkpoint from the SQLite memory database. The workflow uses the earlier question and answer to interpret the follow-up in the same incident context.

Scenario C — Internal evidence is weak

Question: “What are the latest recommended troubleshooting steps for Kubernetes ImagePullBackOff errors?”

When Tavily is configured and internal retrieval is not sufficient, the workflow can rewrite the query, search public sources, assess the returned evidence, and generate a response that includes source information. If Tavily is not configured, that fallback is unavailable.

6. Proposed product components

Component	Actual project technology	Role
API backend	FastAPI, Uvicorn	Chat, upload, ingestion and health endpoints.
Workflow orchestration	LangGraph	Conditional routing, retries, state and checkpoint integration.
LLM provider	Groq via LangChain ChatGroq	Question rewriting, grading, generation and evaluation.
Embedding model	sentence-transformers/all-MiniLM-L6-v2	Local text embeddings; configured dimension is 384.
Vector store	Qdrant local persistent storage	Semantic retrieval over ingested runbook/document chunks.
Application audit DB	SQLite (data/audit.db by default)	Stores question, answer, route, web-use flag, status, trace and sources.
Workflow memory	SQLite LangGraph SqliteSaver	Persists workflow checkpoints and conversational context.
Web search	Tavily API (optional)	External fallback when configured.
Web interface	HTML, CSS, JavaScript, Jinja2	Browser UI for chat and document upload.
Ingestion	PyPDF, TextLoader, python-docx, text splitters	Supports PDF, TXT, MD and DOCX; chunks are indexed in Qdrant.
Packaging	Dockerfile; Docker Compose not confirmed in supplied root	Container image definition exists; deployment status is separate.

7. Knowledge base and document ingestion

RunbookPilot indexes operational documents placed in the project's documents directory and supports upload of PDF, TXT, Markdown, and DOCX files through the application. The ingestion pipeline loads document text, splits it into overlapping chunks, attaches source metadata, and adds the chunks to the configured local Qdrant collection using stable IDs to reduce duplicate inserts during repeat ingestion.

Current sample runbook documents in the supplied project

- **checkout-api-runbook.md** — checkout/API troubleshooting guidance.
- **payments-high-cpu-runbook.md** — payments service high-CPU guidance.
- **deployment-rollback-sop.md** — deployment rollback procedures.

Ingestion flow

1. Load	2. Split	3. Embed	4. Index
Read PDF, TXT, MD or DOCX files.	Create chunks with configured size and overlap.	Generate 384-dimensional embeddings with Sentence Transformers.	Store vectors and metadata in local persistent Qdrant.

8. Persistence and data handling

Data store	Path / mechanism	What it persists
LangGraph checkpoint SQLite	data/langgraph_memory.sqlite	Graph checkpoints and conversation state associated with thread IDs.
Audit SQLite	data/audit.db by default	Question, answer, route, web-search usage, support/usefulness status, trace and source metadata.
Qdrant	data/qdrant by default; collection cloudops_sentinel	Document vectors and payload metadata for semantic search.

Data and security considerations

- Keep API keys in local environment configuration; do not commit secrets or include them in screenshots.
- Treat uploaded runbooks and incident logs as potentially sensitive operational data.
- Local persistence means the data directories must be retained and backed up if the application is moved to another machine or container.
- The supplied project uses local Qdrant storage. Pinecone and hosted Qdrant are not part of the current implementation.

9. User interface requirements and current scope

The existing application provides a browser-based interface served by FastAPI/Jinja templates, client-side JavaScript and CSS, chat requests, document upload, and status/trace/source information exposed by the backend. The design may be extended with richer workflow visualization, session navigation, and operational dashboards, but those should be presented as planned enhancements unless verified in the current UI.

10. API and application surface

Endpoint / function	Purpose	Implementation note
POST /api/chat	Submit an operational question and receive a generated answer.	Runs the Self-RAG workflow and records audit data on successful completion.
POST /api/upload	Upload a supported document for ingestion.	Supported extensions: PDF, TXT, MD, DOCX.
POST /api/ingest	Ingest documents from the configured documents directory.	Confirm the exact request schema in app.py before external integration.
GET /api/health	Health/status response.	Use for local service checks; it does not by itself prove every external dependency is healthy.
Audit helpers	Read recent audit records.	Implemented in src/db.py; availability through UI/API depends on app routes.

11. Expected business value

- Reduce time spent searching across operational documents during incidents.
- Provide practical troubleshooting guidance grounded in retrieved runbook content.
- Improve retrieval quality through relevance grading and query rewriting.
- Reduce unsupported recommendations through support evaluation and answer revision.
- Preserve context across follow-up questions using LangGraph SQLite checkpoints.
- Make uploaded operational documents searchable through the same ingestion pipeline.
- Expose sources and workflow trace details to support inspection and debugging.
- Offer an optional external-search path for knowledge gaps when Tavily is configured.

12. Acceptance criteria for validation

Test	Expected result
Initial ingestion	Running python data_ingestion.py creates/validates the local Qdrant collection and indexes supported files in documents/.
Internal retrieval	A question about an indexed runbook retrieves relevant chunks and includes source information where returned by the workflow.
Conversation memory	A follow-up on the same thread can use checkpointed conversation context.
Web fallback	When TAVILY_API_KEY is configured and internal evidence is insufficient, the workflow can invoke the Tavily path.
Support/usefulness checks	The configured Groq model returns compatible structured decisions; unsupported/unhelpful answers follow bounded revision paths.
Dynamic ingestion	A supported uploaded file is processed and becomes retrievable from the same Qdrant collection.
Audit records	Successful chat results create records in SQLite with trace and source data.
Container startup	Docker image starts the FastAPI app; persistent volumes must be configured for SQLite and Qdrant data.

13. Deployment and operations

The repository includes a Dockerfile based on Python 3.11 slim and launches Uvicorn on port 8080 by default. The application can be containerized for local testing or a cloud host. DigitalOcean is a possible deployment target, but the supplied code alone does not establish that a production deployment is already live.

Suggested deployment sequence

Step	Activity	Verification
1	Configure environment variables for Groq, embedding model, Qdrant path, and optional Tavily.	No secrets are committed; required values load successfully.
2	Install dependencies and run the app locally.	FastAPI starts and health endpoint responds.
3	Run python data_ingestion.py.	Sample documents are indexed in Qdrant.
4	Validate chat, retrieval, memory, sources, upload and audit records.	Check actual behavior and model compatibility; record any known limitations.
5	Build and run the Docker image.	App starts with required environment variables.
6	Configure persistent storage for data/audit.db, data/langgraph_memory.sqlite and data/qdrant.	Data survives container restarts/redeployments.
7	Deploy to a selected cloud host, such as DigitalOcean, if desired.	HTTPS, access control, logs, backups and restart policy are configured.

14. Monitoring, limitations and future improvements

- Review FastAPI/application logs and SQLite audit records when diagnosing failures.
- Add explicit readiness checks for Groq credentials, Qdrant collection access, embedding model availability, and optional Tavily configuration.
- Verify the configured Groq model supports the structured-output mode used by the Self-RAG nodes; provider/model mismatch can cause chat failures.
- Consider authentication, role-based access, upload limits, file validation, secret management, and HTTPS before production exposure.
- Consider managed database/vector services only if operational needs justify moving beyond the current local SQLite and Qdrant setup.

15. Forward-deployed engineering perspective

RunbookPilot is not just an LLM chat interface. Its intended value comes from connecting an operational problem to the existing runbook corpus, indexing that knowledge, retrieving and grading evidence, revising weak answers, preserving session context, and returning source-aware guidance through an application that can be tested and packaged for deployment.

End-to-end delivery path
Operational question → FastAPI → LangGraph workflow → Local Qdrant retrieval → Evidence grading → Optional query rewrite / Tavily fallback → Groq response generation → Support & usefulness checks → Answer + sources → SQLite audit and checkpoint persistence

Prepared from the supplied RunbookPilot/CloudOps Sentinel source repository. Features described as optional, planned, or requiring validation are intentionally distinguished from confirmed source-code components.